

Professional User, Operator, Maintainer, Developer, Verification, and Recovery Manual

Proprietary Commercial Edition | Release date: 27 July 2026 | Documentation build: source-audited
1.7.6 edition

Document Control

Product	DXR 1.7.6 Quine AIO
License	Proprietary - Copyright (c) 2026 Alex Moraru; third-party components retain their own licenses
Scope	Installation, operation, inputs, outputs, architecture, maintenance, repair, verification, release, and recovery
Verified host	Linux x86_64, Python 3.12.3, GCC 13.3.0
Test result	132 passed, 0 failed, 0 skipped; 17-gate release matrix PASS; 11/11 tamper controls rejected; two-generation fixed point proven; 25/25 instrumented desktop and template checks

This manual is generated from the DXR 1.7.6 source tree, live CLI inspection, and its recorded verification evidence. It distinguishes executed results from platform capabilities that were not exercised on the verification host.

Contents

1. Part I — Product and User Manual
2. Part II — Architecture and Internal Design
3. Part III — Build, Packaging, and Release
4. Part IV — Quine and Machine Verification
5. Part V — Maintenance, Repair, and Recovery
6. Part VI — Security, Licensing, Limitations, and Reference
7. Appendix A — Verification Evidence
8. Appendix B — Schematics

Part I — Complete Owner and User Manual

This manual covers Dextr 1.7.6 as shipped. It is written for two readers at once: the owner who builds, validates, and ships Dextr packages, and the end user who installs and works in the Dextr desktop or command line. Every command, flag, panel, and limit described here exists in the shipped source tree; nothing in this document describes planned or hypothetical behavior.

Companion documents: README.md, docs/USAGE.md, docs/ARCHITECTURE.md, docs/BUILD.md, CHANGELOG.md, TEST_REPORT.md, RELEASE_VALIDATION.md, THIRD_PARTY_NOTICES.md, and LICENSE.

1. Introduction

Dextr is a standalone local coding workspace. It combines a dark desktop application, a recursive local agent mesh, a deterministic native routing engine (NeuroRoute), an approval-gated tool layer, a validation and bounded repair loop, local memory, and a reversible self-build registry — all inside one self-contained process tree.

Dextr is its own application. It does not start, attach to, inspect, modify, or exchange data with any external coding application or hosted coding API.

Optional model traffic is restricted to a canonical numeric `**loopback`

`address**` (127.0.0.1 or ::1 after canonicalization), with proxies disabled and redirects rejected, for an optional local model

runtime that the user installs and configures. Approved tool commands run only through the reported restricted-process backend; unsupported command-execution platforms fail closed.

Who Dexr is for

- **End users** who want a private, offline-first coding assistant that works with no account, no subscription, and no cloud dependency. Dexr runs fully in deterministic mode out of the box; adding a local model is optional.
- **Owners / distributors** who build and ship the Linux bundle, the macOS app, and the source/AIO packages, and who need reproducible native engine builds for six platform targets.

The offline-first philosophy

Four rules define the product boundary:

9. **Works with nothing installed.** Dexr's deterministic mode needs no model weights, no model runtime, and no network. The deterministic engine (NeuroRoute plus templates) produces answers, routes, and validations on any machine with Python 3.10+.
10. **Model use is opt-in and local.** A local model runtime with a chat-completions-compatible loopback API can be added at any time. Model weights are never bundled and never downloaded automatically — a download is always an explicit user action, from the model's official source, under the model author's own license.
11. **Bounded autonomy.** Recursion depth, node counts, tool calls, batch width, repair attempts, and routing-policy adaptation all have hard caps. Dexr never claims unbounded self-improvement.
12. **Deterministic by construction and verified per target.** The C and Python implementations use a fixed operation order and deterministic arithmetic. The bundled verifier proves equivalence and native binary reproducibility only on the host/target it actually executes. The current release report records Linux x86_64; other targets remain unverified until the same gates are run there.

What "recursive", "learning", "healing", and "self-building" mean

These words are deliberately bounded in Dextr:

- **Recursive** — a task can be decomposed into a limited tree of specialist agent nodes (depth, nodes, and tool calls are capped).
- **Learning** — Dextr stores validated experiences, exact replay entries, cached tool results, and small bounded routing biases. It does not train or alter model weights.
- **Healing** — failed generated work can be validated and repaired for a limited number of attempts (0-4). A task that exhausts its repair budget stays failed; Dextr never falsifies validation status.
- **Self-building** — Dextr can generate, validate, version, activate, and roll back isolated Python runtime modules stored in the user state directory. It never overwrites its own installed application source.

2. Installation & first run

2.1 Requirements

Minimum:

- Python 3.10 or newer;
- Tk support (``tkinter``) for the desktop interface;
- a writable user state directory.

Recommended:

- Git, for repository fingerprints and the Changes (diff) panel;
- a C11 compiler (``cc`/`gcc`/`clang``) and Make, to build the native

NeuroRoute engine in place;

- any local model runtime with a chat-completions-compatible loopback API, for model-backed agents.

Dexr works in deterministic mode without any of the recommended items.

Verify Tk before installing:

```
python3 -c "import tkinter; print(tkinter.TkVersion)"
```

2.2 Installing from source

```
python3 install.py
```

install.py supports two modes:

Flag	Effect
--- ---	
`--quick`	Build/import only; skips the complete test matrix (runs `scripts/bootstrap.py`).
`--build-macos-app`	After validation, also creates `dist/Dexr.app`.
(no flags)	Runs the full release check matrix (`scripts/run_all_checks.py`).

On success the installer prints Dexr installation validated.

2.3 First run

From the source tree:

```
./run.sh /path/to/project          # Linux/macOS launcher script
# or:
python3 main.py desktop --workspace /path/to/project --start-backend
```


On macOS, Dextr.command is provided for double-click launch. A

Dextr.desktop entry is included for Linux desktop environments.

From the Linux x86_64 bundle:

```
unzip Dextr-1.7.6-linux-x86_64.zip
cd Dextr-1.7.6-linux-x86_64
chmod +x Dextr
./Dextr
```

To preselect a project:

```
DEXR_WORKSPACE="$HOME/Projects/example" ./Dextr
```

From the macOS package: unzip, move Dextr.app to Applications if desired, and open it. The app is unsigned; macOS may require Control-click → Open on first launch. On first start the app attempts to build the small NeuroRoute dynamic library with the compiler on the Mac; if no compiler is present, Dextr starts anyway with the bit-exact Python router.

2.4 Directory layout (source tree)

main.py	CLI entry point (desktop, run, doctor, policy, prompt, agent, self-test)
install.py	install + validation entry point
run.sh / Dextr.command	convenience launchers
app/	desktop UI, orchestrator, session store, agent stack
agents/	recursive agent mesh runtime
engine/	NeuroRoute C source, Python router, Octopus registry, local-model client
brain/	deterministic core, adaptive policy
batching/	bounded batch planner, executor, validated tool cache
tools/	workspace-confined tool catalog
integration/	validation runner, workspace fingerprinting, locking, and restricted-process sandbox

selfbuild/	reversible generated-module registry
observer/	redacted event journal
replica/	deterministic compatibility agent loop
owner/	Owner Engine build console (python3 -m owner)
quine/	canonical capsule, fixed-point service, content-addressed promotion and rollback
scripts/	bootstrap, bundlers, local-model setup, release checks
tests/	unit and integration tests
state/	default runtime state (source-tree installs)
docs/	documentation (this manual, USAGE, ARCHITECTURE, BUILD)

2.5 State directory

Dexr stores its database, journals, caches, policies, and self-build modules outside your project and outside the installed app:

Platform Default state directory
--- ---
Linux `\${XDG_STATE_HOME:--/.local/state}/Dexr`
macOS `~/Library/Application Support/Dexr`
Windows (source execution) `%LOCALAPPDATA%/Dexr`

DEXR_STATE_DIR overrides the location. Inside it, each workspace gets an isolated subdirectory keyed by a SHA-256 hash of its absolute path (workspaces/<hash>/), containing the core state, adaptive policy, replay cache, local-model journal, event journal (events.jsonl), and self-build data. The cross-workspace database is dexr.sqlite3 in the state root.

Deleting the state directory removes all Dexr history, caches, and generated modules. Project files are never touched by that action.

3. The desktop app

The desktop (app/desktop.py) is a single dark-themed Tk window titled Dextr (default 1500×920, minimum 1120×700). This chapter tours every element, top to bottom, left to right.

3.1 Top bar

```
| Control | Action |
|---|---|

| **New task** (accent button) | Creates a new thread in the current workspace. Disabled while a turn
is running. |

| **Doctor** | Runs the full readiness report (see §4.3) and shows it in the Activity panel. |

| **Open project** | Opens a directory picker and switches workspace, starting the backend for the new
project. |

| ** Download AI Engine..." (accent button) | Opens the engine download dialog (§3.7). |
```

3.2 Workspace bar

Below the top bar, left to right:

- ****Workspace path**** — the absolute path of the open project.
- ****Route: ...**** — the NeuroRoute strategy chosen for the most recent turn
(one of the eight strategies in §6), or — before the first turn.
- ****Mode**** combobox — selects the runtime mode: `auto`, `local-model`, or deterministic (§5.1). Applied to the next turn.
- ****Engine: ...**** — backend status: `not started`, `ready`, the active backend name, Local model: ready / Local model: unavailable, or Engine: deterministic-ready.

3.3 Left pane: Files and Threads

- ****FILES**** — a tree of the workspace, directories first, sorted case-insensitively. Symlinks and the directories .git, .dextr,

__pycache__, .venv, venv, node_modules, dist, and build are

hidden. The tree is capped at depth 12 and 3000 entries. Selecting a file opens it in the Editor tab.

- **THREADS** — persistent task threads for this workspace, newest state shown in the status column. Selecting a thread loads its transcript into the Agent tab.
- **+** button — new thread (same as **New task**).
- **Refresh** button — reloads files, threads, and the Git diff.

3.4 Center: Agent tab

The Agent tab holds the transcript (read-only conversation view) and the composer:

- Transcript headers are ``YOU``, ``DEXR``, ``SYSTEM``, and `DEXR · VALIDATED REPLAY` (when an exact validated replay is served from the cache for an unchanged workspace).
- The **composer** is a 5-line text box for your prompt.
- **Validate** checkbox (default on) — run project validation after model-backed turns.
- **Replay validated cache** checkbox (default on) — allow exact validated replay when prompt and workspace fingerprint match a previous success.
- **Send**  (accent button) — submits the prompt. Disabled while busy.
- **Stop** (danger button) — requests cancellation of the current turn.

Enabled only while a turn runs.

3.5 Center: Editor tab

- Header shows the selected file's workspace-relative path (or No file selected).
- **Save** — asks for confirmation (``Save changes to <path>?``), writes the prior content to `.dexr-backups/<name>.<timestamp>.bak` inside the

workspace, then saves atomically via a temporary file and rename.

- ****Reload**** — re-reads the file from disk, discarding unsaved edits.
- The editor is monospace with undo support. It opens UTF-8 text files up to

2 MB; larger files and binary files are refused with an explanatory

dialog. Saves outside the workspace are rejected.

3.6 Right pane: Activity, Changes, Memory, Tools

- ****Activity**** — structured JSON events for the session: routes, agent start/finish, batch stages, tool cache hits, command output, validation results (with per-check exit codes and bounded output), doctor reports, and model health reports.
- ****Changes**** — the workspace's unstaged Git diff (git diff --no-ext-diff --), or No unstaged Git changes., or an explanation when Git is unavailable.
- ****Memory**** — a JSON snapshot of the adaptive routing policy (biases and revision), event journal counts, the twelve Octopus arms, the thread count, and the configured local model tag.

- ****Tools**** — eight buttons:

| Button | Effect |

|---|---|

| Run project validation | Runs discovered validators (§7.5) and reports pass/fail in Activity. |

| Refresh Git diff | Reloads the Changes panel. |

| Show policy and Octopus | Reloads the Memory panel. |

| Build reversible module... | Prompts for a module name and Python source; validates, versions, and promotes it through the self-build registry (§7.7). |

| Check local model | Probes the configured loopback endpoint and model tag; updates the engine status. |

| Set up local model... | Asks for your runtime's model tag, confirms, then starts the runtime and pulls the tag (§5.3). |

| Open project folder | Opens the workspace in the OS file manager. |

| Owner: build engines... | Runs the Owner Engine console (status + build) and reports N/6 engine targets built (§9.3). Absent builds show "not installed in this build". |

A footer note in the Tools tab states the isolation boundary: loopback local-model runtime or deterministic engine only; writes and commands are approval gated; recursion, batching, repair, cache, and self-build are bounded.

3.7 The Download AI Engine dialog

↓ Download AI Engine... opens a modal dialog titled Download AI Engine:

- A header explains that AI models are downloaded from their official sources and licensed separately by their authors, and that Dextr works offline without any download (deterministic mode).
- A **runtime status line** reports whether a local model runtime was detected, and whether it is running with a model present.
- A free-text field: **Enter the model tag from your local model runtime** (prefilled with DEXR_LOCAL_MODEL or dextr-local).
- A table of the four **generic Dextr engine tiers** (§5.2): tier name, parameter count, approximate size, and the license note.
- **Download** — starts the runtime and pulls the entered tag from its official source, streaming progress into the status line. The button is disabled while a download runs.
- **Use offline deterministic mode** — closes the dialog; no download occurs.

Dextr never picks a model for you and never downloads without the Download button (or the explicit setup actions in §5.3).

3.8 The approval dialog

Whenever the agent mesh wants to write a file or run a command, a modal Dextr approval dialog appears showing the reason, the exact command, the

target path, and the working directory. Yes allows the action, No declines it, Cancel cancels the turn's request. If the dialog is ignored for 600 seconds the request is treated as cancelled. Declined writes do not happen; the agent is told and must continue without them.

3.9 Keyboard shortcuts

Shortcut	Action
--- ---	
<code>`Cmd/Ctrl + Return`</code>	Send the composer prompt
<code>`Cmd/Ctrl + N`</code>	New task thread
<code>`Cmd/Ctrl + S`</code>	Save the current editor file (with confirmation + backup)
<code>`Escape`</code>	Stop the current turn

(Cmd on macOS, Ctrl on Linux and Windows source runs; both bindings are registered on every platform.)

3.10 Status bar

The bottom status bar shows the last status message (green on success, muted otherwise) on the left and the running Dexr version on the right.

3.11 Typical workflow

13. ****Open project**** → pick your repository. The engine starts in the background; the status bar shows the chosen backend.
14. ****New task**** → a thread appears in the Threads list.
15. Type a prompt in the composer → ****Send**** (``Cmd/Ctrl+Return``). Watch the Route label, the Activity panel, and agent status updates.
16. Answer any ****approval dialogs**** for writes or commands.
17. Review ****Changes**** (Git diff) and the transcript. Edit files directly in the Editor tab if needed; saves are confirmed and backed up.
18. ****Run project validation**** (Tools tab) before accepting generated

changes; with the Validate checkbox on, validation also runs automatically after model turns, and failures trigger bounded repair.

19. ****Save**** your edits; the previous content is always in ``.dextr-backups``.

On first launch in auto/local-model mode without a model present, the status bar hints: `*"Download AI Engine... for local LLM mode — or keep using deterministic offline mode."*` In deterministic mode no hint is shown.

4. The command line

All CLI commands run through `main.py`. Running `python3 main.py` with no subcommand is equivalent to `python3 main.py desktop`. Global error behavior: `KeyboardInterrupt` exits 130; other exceptions print Dextr error: `<message>` to `stderr` and exit 1.

4.1 ``desktop`` — launch the GUI

```
python3 main.py desktop [--workspace PATH] [--start-backend]
```

```
                [--backend {auto,local-model,deterministic}]
```

```
                [--local-model-url URL] [--local-model TAG]
```

```
                [--recursive-depth 0-8]
```

Flag	Default	Meaning
<code>--workspace`</code>	current directory	Project to open.
<code>--start-backend`</code>	off	Probe/start the backend immediately at launch.
<code>--backend`</code>	<code>`auto`</code>	Initial mode for the Mode combobox.
<code>--local-model-url`</code>	<code>`\$DEXR_LOCAL_MODEL_URL`</code> or <code>`http://127.0.0.1:11434/v1`</code>	Loopback endpoint.
<code>--local-model`</code>	<code>`\$DEXR_LOCAL_MODEL`</code> or <code>`dextr-local`</code>	Model tag.
<code>--recursive-depth`</code>	<code>`3`</code>	Agent mesh depth cap, 0-8.

4.2 `run` — one managed task, no window

```
python3 main.py run --workspace /path/to/project \
  --prompt "Add tests for the parser" --backend auto \
  --recursive-depth 3 --max-repair-attempts 2
```

Flag	Default	Meaning
--- --- ---		
`--workspace`	*(required)*	Project directory.
`--prompt`	*(required)*	Task text.
`--backend`	`auto` `auto`, `local-model`, or `deterministic`.	
`--local-model-url`, `--local-model`	env / defaults	As above.
`--recursive-depth`	`3`	0-8.
`--no-validation`	off	Skip post-run validation.
`--no-cache`	off	Disable the validated replay cache for this run.
`--max-repair-attempts`	`2`	Bounded repair budget, 0-4.

Output is the full result as indented JSON; exit code is 0 when

result.success is true, else 1. Shape:

```
{
  "success": true,
  "thread_id": "...",
  "backend": "local-model",
  "model": "dextr-local",
  "route": "test",
  "scores": {"function": ..., "class": ..., "test": ..., "refactor": ...,
            "import": ..., "print": ..., "explain": ..., "fallback": ...},
  "response": "...",
  "cached": false,
  "validation": [{"name": "Python tests", "argv": ["..."], "returncode": 0,
                  "stdout": "...", "stderr": "", "duration_ms": 812.4,
```

```

    "timed_out": false, "passed": true}],

    "attempts": 1,

    "recursive_agents": {"success": true, "nodes": 3, "tool_calls": 5, ...},

    "workspace_changed": true,

    "policy_revision": 17
}

```

A replay-cache hit returns backend: "validated-replay", cached: true, attempts: 0. Deterministic runs return backend: "deterministic" and an empty validation list.

4.3 `doctor` — readiness and boundary report

```
python3 main.py doctor [--workspace PATH] [--local-model-url URL] [--local-model TAG]
```

Prints a JSON report. Exit code is 0 when the native router is loaded, otherwise 1; the bit-exact Python router remains available. DXR 1.7.6 adds an explicit `command_sandbox` capability record so operators can see whether network, process, and filesystem-write isolation are actually enforced on the current host.

```

{
  "app": "Dexr",

  "version": "1.7.6",

  "native_router": true,

  "neuroroute_extended": {"available": true, "router_version": "0x90000"},

  "local_model": {"provider": "proxy-free redirect-free loopback local runtime", ...},

  "command_sandbox": {
    "available": true,

    "backend": "linux-bubblewrap | linux-user-net-pid-ipc-namespace | macos-seatbelt",

    "network_isolated": true,

    "filesystem_write_isolated": true | false,

    "process_isolated": true
  },

  "runtime_boundary": {

```

```

"network": "numeric-loopback-only model traffic; tool network depends on command_sandbox",
"workspace": "file tools confined; command filesystem enforcement reported",
"writes": "approval gated; commands allowlisted, capped, sandboxed, fail closed",
"self_build": "runtime modules only; application source is not overwritten"
}
}

```

4.4 `policy` — bounded routing policy

```

python3 main.py policy show --workspace /path/to/project
python3 main.py policy reset --workspace /path/to/project

```

show prints the policy snapshot (biases per strategy, revision, and bounded update history); reset clears it. Both exit 0.

4.5 `prompt` — direct deterministic generator

```

python3 main.py prompt "def fibonacci(n):"
python3 main.py prompt "class Stack:" --json

```

Argument/flag	Default	Meaning
`text`	*(required, positional)*	Prompt for the deterministic generator.
`--model`	`dexr-deterministic`	Model label mixed into routing features.
`--json`	off	Print the full result JSON instead of just the generated text.

4.6 `agent` — deterministic compatibility recursive agent

```

python3 main.py agent --prompt "def add(a, b): then class Calculator:" --max-depth 2

```

Runs the deterministic compatibility agent loop (no model required) and prints its result as JSON. --max-depth accepts 0–4 (default 2).

4.7 `self-test` — full release matrix

```
python3 main.py self-test
```

Runs scripts/run_all_checks.py inside a fresh temporary DEXR_STATE_DIR and restores the caller environment afterward. The 1.7.6 matrix contains 17 gates:

20. release-file inventory;
21. forced Python syntax compilation;
22. native host-library build;
23. strict C runtime test;
24. AddressSanitizer and UndefinedBehaviorSanitizer test;
25. the full Python unit and integration suite;
26. a second plain python3 -m unittest discover gate from a fresh state directory;
27. restricted-process isolation smoke;
28. local-model recursive end-to-end smoke;
29. desktop GUI smoke when a display backend is available;
30. macOS app-bundle structural generation;
31. Linux x86_64 bundle generation and runtime checks on matching hosts;
32. two-generation payload-quine fixed-point proof;
33. shell launcher syntax;
34. source-isolation and security-marker scan;
35. CLI runtime smoke.

Success prints ALL DEXR CHECKS PASSED. Repeated clean-extraction campaigns run the complete matrix end to end, with the Python suite executed twice per matrix. Individual platform-dependent gates must still report skips or unavailable capabilities honestly.

For manual plain discovery outside self-test, use a fresh DEXR_STATE_DIR. Reusing a state directory can legitimately change replay-dependent expectations because validated replay is persistent product behavior.

5. AI engines and models

5.1 Runtime modes

Mode	Behavior
---	---
`auto` (default)	Use the configured local model when the runtime is reachable and the model tag is present; otherwise use the deterministic engine.
`local-model`	Require the loopback model path; if unavailable, emit a `backend_fallback` warning event and fall back safely to deterministic.

```
| `deterministic` | Use NeuroRoute routing plus deterministic templates only. No model contact is attempted. |
```

If a model turn fails mid-run because the local runtime errors, Dextr falls back to the deterministic engine for that turn without contacting any service.

5.2 Engine tiers

Dextr ships no model weights and no third-party model identifiers. The catalog is four generic size classes; the tag field is intentionally empty — you fill it in with the model tag from your own local model runtime:

```
| Tier | Name | Parameters | Approx. size | License |
| --- | --- | --- | --- | --- |
| compact | Dextr Engine – Compact 7B | 7B | ~5 GB | See the model author's license at the download source |
| balanced | Dextr Engine – Balanced 14B | 14B | ~9 GB | See the model author's license at the download source |
| standard | Dextr Engine – Standard 20B | 20B | ~12 GB | See the model author's license at the download source |
| performance | Dextr Engine – Performance 32B | 32B | ~20 GB | See the model author's license at the download source |
```

Tiers are guidance for matching a model to your hardware (memory and disk).

Any compatible tag your runtime provides works with any tier.

Print the catalog any time:

```
python3 scripts/setup_local_model.py --list          # table
python3 scripts/setup_local_model.py --list --json   # JSON
```

5.3 Setting up a local model

36. Install any local model runtime that exposes a chat-completions-compatible

loopback API.

37. Run the helper:

```
python3 scripts/setup_local_model.py --start --pull --model dextr-local
```

Flags: --model TAG (default \$DEXR_LOCAL_MODEL or dextr-local),
 --url URL (default \$DEXR_LOCAL_MODEL_URL or http://127.0.0.1:11434/v1),
 --runtime PATH (explicit runtime executable; otherwise auto-detected on
 PATH and in standard install locations), --start, --pull, --json,
 --list. Exit codes: 0 ready, 2 no runtime found, 3 not ready, or the
 runtime's pull exit code on download failure.

38. Equivalent desktop path: ****Tools → Set up local model...**** (guided) or
 ↓ **Download AI Engine...** (tier table + progress).

Downloads always come from the model's official source via your runtime;
 each model is licensed separately by its authors, and that license is shown
 at the download source. Pulling a model is never automatic.

5.4 Environment variables

Variable	Default	Purpose
---	---	---
<code>`DEXR_LOCAL_MODEL`</code>	<code>`dextr-local`</code>	Model tag sent to the loopback runtime.
<code>`DEXR_LOCAL_MODEL_URL`</code>	<code>`http://127.0.0.1:11434/v1`</code>	Endpoint base URL. Loopback hosts only.
<code>`DEXR_LOCAL_MODEL_TIMEOUT`</code>	<code>`180`</code>	Per-request timeout in seconds.
<code>`DEXR_LOCAL_MODEL_REASONING`</code>	<code>`medium`</code>	Reasoning effort hint: <code>`low`</code> , <code>`medium`</code> , or <code>`high`</code> (other values are not sent).
<code>`DEXR_LOCAL_MODEL_MAX_TOKENS`</code>	<code>`4096`</code>	Maximum output tokens per model call.
<code>`DEXR_STATE_DIR`</code>	platform default (§2.5)	Overrides the state directory.
<code>`DEXR_WORKSPACE`</code>	—	Used by launchers to preselect a project.

5.5 The loopback-only security rule

The 1.7.6 local-model client is numeric-loopback-only after canonicalization. It accepts HTTP or HTTPS only, rejects remote hosts, credentials, query strings, fragments, invalid ports, backslashes, control characters, unsafe request paths, and unsupported HTTP methods.

- localhost is converted to 127.0.0.1 before requests, avoiding hosts-file or DNS rebinding of the friendly name.
- Environment proxy variables are ignored through an explicit empty proxy handler.
- HTTP redirects are rejected; the final response URL must equal the requested URL.
- JSON request bodies are capped at 8 MiB and responses at 16 MiB.
- Request timeout is clamped to 0.1-600 seconds; model names are length- and control-character-validated.
- No remote override flag or API-key transport exists.

These controls protect the endpoint boundary; they do not turn a remote endpoint into a supported or safe configuration.

6. How routing works — the NeuroRoute engine

Every prompt entering Dextr is scored by the NeuroRoute router. Routing is purely local and fully deterministic.

6.1 The 24-feature input vector

For a prompt and model tag, the router builds 24 float features:

- ****16 digest features**** — bytes 0–15 of SHA-256(model_tag + NUL + prompt), each divided by 255.
- ****8 lexical features**** — presence flags for ``def``, ``class``, ``test``, `refactor`, `import`, `print (1.0/0.0)`, plus prompt length capped at 1000 characters and newline count capped at 20, both normalized to [0, 1].

6.2 The 24→32→8 network

The features pass through a two-layer perceptron: 24 inputs → 32 hidden units → 8 outputs. Weights and biases are generated from a fixed splitmix64 seed,

in a fixed order (all weights row-major, then biases, per layer), with a fixed accumulation order. The eight outputs are scores for the eight strategies:

function, class, test, refactor, import, print, explain, fallback.

Clear lexical intent overrides the network: a prompt containing `def /function`, `class`, `unit test/test for`, `refactor`, starting with `import`, or containing `print` routes directly to the matching strategy. Otherwise the highest-scoring strategy wins. The chosen strategy selects the deterministic template family and is recorded as the turn's route (shown in the desktop Route label and in results).

6.3 The Padé deterministic activation

The hidden activation is a Padé (7/6) tanh approximant evaluated in pure IEEE-754 double arithmetic — no libm, no fast-math, no FMA contraction (`-ffp-contract=off`). Inputs beyond ± 4.97 saturate to exactly ± 1.0 . Because every operation is correctly rounded and the order is fixed, scores are identical on Linux, macOS, and Android, on x86_64 and arm64.

6.4 Bit-exact Python fallback

When the native shared library is absent or fails to load, Dexr uses a pure-Python mirror of the C router (`engine/octopus.py`). The fallback applies float32 rounding after every 32-bit operation (each float32 op is exact in double and rounded once, matching C float semantics) and evaluates the same Padé activation with the same constants and order. The test suite (`tests/unit/test_router_equivalence.py`) compares native and fallback over hundreds of prompt samples with zero bit mismatches — routing decisions

do not depend on whether the native engine loaded. The doctor field `native_router` tells you which path is active; results report the backend as `native-c-router` or `python-router-fallback`.

6.5 The extended `nr_*`` graph router

`libneuroroute` also carries the extended NeuroRoute API (engine/c/neuroroute/nr_neuro.h, Python bindings and identical fallbacks in engine/neuroroute_ext.py), preserved from the original NeuroRoute design:

- **Maximum-weight graph routing** — up to 1024 nodes (`NR_MAX_SYSTEM_NODES`). Each node has a local fallback address and weighted outbound edges, traversed in insertion order. `nr_route_synaptic_signal` selects the highest-weight outbound edge (first match wins ties); with no edges it returns the node's fallback.
- **Activation threshold** — inputs below `NR_ACTIVATION_THRESHOLD`` (100) return zero: the signal is too weak to route.
- **Atomic spike counters** — every routed signal increments a per-edge spike counter, queryable with `nr_get_edge_spike_count`, giving a deterministic usage profile of the graph.
- **Spatial pool** — a sparse pool of up to 256 chunks (`NR_MAX_SPATIAL_CHUNKS`), each addressed by a 256-bit coordinate (4 × 64-bit words) with a density score. `nr_allocate_spatial_memory` returns the page of the configured chunk with the minimum Hamming distance to the query vector (first best match wins).

doctor reports this as `neuroroute_extended.available` and the library `router_version` (extended API from 0x00090000).

6.6 Adaptive policy — bounded bias only

On top of the fixed network, `brain/adaptive_policy.py` keeps one small bias per strategy. After each turn, the chosen strategy's bias is nudged by $\text{learning_rate} \times \text{reward}$ (positive on success, negative on failure), all biases decay by $\times 0.999$ per update, and every bias is clipped to ± 3.0 . The policy is persisted per workspace with a monotonically increasing revision and a bounded history. This is a bounded bias layer — it can reorder close-scoring routes over time but can never rewrite the network. Inspect or clear it with `python3 main.py policy show|reset`.

6.7 Validated replay cache

Before any engine runs, Dextr checks its experience store for the **exact same normalized prompt with the exact same workspace content fingerprint** and a passing validation record. Only then is the stored answer replayed (transcript header `DEXR · VALIDATED REPLAY`; result backend `validated-replay`). Any file change anywhere in the workspace changes the fingerprint and forces a fresh run. Disable per turn with the **Replay validated cache** checkbox or `--no-cache`.

7. The agent system

7.1 The recursive agent mesh

In local-model mode, tasks run through a bounded recursive agent mesh (`agents/recursive_runtime.py`). The model answers each node with one of three JSON actions:

39. ``{"action": "final", "text": "...}"`` — finish this node with an answer.
40. ``{"action": "delegate", "tasks": [{"role": "...", "goal": "...}]}`` — spawn child agents for genuinely separable subwork.

41. ``{"action":"batch","calls":[{"id":..., "tool":..., "arguments":{...}, "depends_on":[...]}]}``

— request a bounded batch of tool calls with explicit dependencies.

Roles and their charters:

Role	Charter
planner	Decompose the goal into the smallest useful independent investigations and stages.
repository_analyst	Inspect concrete repository evidence; prefer read-only tools; report exact paths and constraints.
implementer	Produce the smallest correct implementation plan or change; preserve unrelated behavior.
test_designer	Define focused syntax, unit, integration, and regression checks.
critic	Find correctness, safety, portability, concurrency, and validation gaps; never invent failures.
repair	Use verified failures only; bounded corrective plan; no unrelated changes.
synthesizer	Combine child results into one precise answer without duplication.
agent	General role: solve the goal with evidence, bounded tools, delegation, validation.

Budgets (defaults; RecursiveAgentLimits): depth ≤ 3 (CLI 0-8, hard cap 6 internally), nodes ≤ 16 (cap 64), tool calls ≤ 24 (cap 128), batch width ≤ 4 (cap 8), context $\leq 60\,000$ chars, output ≤ 4096 tokens. Duplicate (role, goal) signatures are rejected, so the mesh cannot cycle. Independent read-only work runs in parallel up to the batch width; writes never run in parallel.

7.2 Tool catalog and workspace confinement

Four tools (tools/catalog.py):

Tool	Writes?	Description
<code>`read_file`</code>	no	Read UTF-8 text inside the selected workspace.
<code>`write_file`</code>	yes	Write UTF-8 text inside the selected workspace. Approval-gated.

| ``list_files`` | no | List one workspace directory. |

| ``run_command`` | command side effects possible | Run one allowlisted bare executable through the restricted-process runner; shell=False; approval-gated by the agent layer. |

All paths resolve inside the selected workspace; traversal outside it is

rejected. Commands use shell=False and an absolute resolved executable, then pass through

the restricted-process backend. Arguments, output, time, processes, open files, and generated file sizes are capped. Results containing

sensitive-looking text are never written to the shared cache.

7.3 The approval gate

Every write and every non-read-only command passes through the approval

callback. On the desktop this is the modal dialog of §3.8 with a 600-second

timeout. Without a callback (headless runs), write requests are declined by

default. The agent mesh never has unrestricted filesystem or shell access.

7.4 Validation

integration/validation.py discovers Python syntax and unittest commands or accepts direct custom argv. Shell operators, redirects, pipes, command separators, and newlines are rejected before execution.

- Every validator runs through integration/secure_process.py, never through a shell.
- The child receives isolated HOME, TMP, XDG config/cache/data directories and deterministic Python/locale settings.
- Secret-like environment names and all proxy variables are removed. Additional environment values cannot reintroduce them.
- Executable resolution is explicit; argv is limited to 64 KiB; output is tail-bounded; timeouts terminate the whole process group.
- The process guard applies CPU, address-space, file-size, open-file, process-count, and core-dump limits where the OS exposes them.
- Each result records sandbox backend, network isolation, and filesystem-write isolation. A check passes only with return code 0 and no timeout.

7.5 The validation & repair loop

After a model turn, enabled validation runs. On failure, Dexr builds a

repair prompt from the verified failure text (bounded to the last

20 000 characters, 4 000 in events) and runs the mesh again in the repair role, up to `--max-repair-attempts` (default 2, range 0–4). When the budget is exhausted the task is marked failed — validation status is never falsified. Successful turns store a validated experience (prompt, response, route, workspace fingerprint) and nudge the routing policy.

7.6 Patch engine, editor saves, and backups

File mutations are always recoverable:

- The desktop **Editor → Save** path asks for confirmation, copies the previous content to `.dexr-backups/<filename>.<timestamp>.bak` inside the workspace, then writes atomically (temp file + rename).
- Agent writes go through the approval gate (§7.3), so no file changes without an explicit per-action decision.

7.7 Session store, memory, and self-build

- **Session store** (``app/session_store.py``, SQLite at `<state>/dexr.sqlite3`): projects, threads, messages, events, and validated experiences. Threads survive restarts.
- **Event journal** (``observer/event_journal.py``, ``events.jsonl`` per workspace): every event is redacted before writing — common secret fields are stripped — and counted for the Memory panel.
- **Self-build registry** (``selfbuild/registry.py``): generated Python modules are validated (ast.parse + py_compile + optional declared checks) in an isolated candidates/ area, versioned by `<timestamp>-<sha256-prefix>`, promoted atomically into versions/, and activated through a manifest pointer in active/. rollback restores the previous version; every transition is journaled to `builds.jsonl`. Names must be valid Python identifiers. Generated modules live in the user state

directory — the installed application source is never overwritten.

8. Determinism & reproducibility

8.1 What is guaranteed

42. **Routing is designed for platform stability.** The Padé (7/6) activation uses fixed IEEE arithmetic; builds use `-fno-fast-math` and `-ffp-contract=off`. A platform is considered verified only after its native and fallback tests have actually run there.
43. **Native and fallback routers are compared bit-for-bit on the tested host.** `tests/unit/test_router_equivalence.py` checks exact output rather than a tolerance. The current verification report records the environment in which that test was executed; it does not convert that host result into an unexecuted cross-platform claim.
44. **Workspace fingerprints are content-based.** Replay decisions depend on actual workspace content, not timestamps.
45. **Engine binary reproducibility is proven per executed target.** The verifier performs two independent clean builds, runs the native tests, and compares exact binary hashes. `BUILD_INFO.json` lists only targets actually proven in the current environment. Prebuilt artifacts for other targets are retained but are not newly certified by this report.

8.2 Bit-exactness proof summary

The C router and the Python fallback perform the identical sequence of IEEE-754 operations: `splitmix64` weight generation, single-rounding casts to `binary32`, fixed-order `float32` accumulation, and the same Padé rational function with identical constants and saturation. Because each `float32`

operation is exact when computed in double and rounded once, the Python struct-based rounding reproduces C float semantics bit-for-bit; and because no libm or fused operations appear anywhere, no platform library variation can enter. The equivalence test asserts bit equality, not tolerance.

8.3 Reproducible engine builds

Target	Artifact	Current quine-release status
linux-x86_64	`libneuroroute.x86_64.so`	rebuilt twice and byte-verified in the recorded environment
linux-aarch64	Linux aarch64 prebuilt	retained; not executed in this environment
macos-x86_64	macOS x86_64 prebuilt	retained; not executed in this environment
macos-arm64	macOS arm64 prebuilt	retained; not executed in this environment
android-arm64	Android arm64 prebuilt	retained; not executed in this environment
android-x86_64	Android x86_64 prebuilt	retained; not executed in this environment

9. Building & shipping (owner)

This chapter is for the owner building release artifacts.

9.1 Make targets (host build)

From engine/c/neuroroute/:

```
make          # libneuroroute.so (Linux) or libneuroroute.dylib (macOS)
make test     # builds + runs test_router and test_nr_neuro
              # expect ROUTER_TEST_PASS and NR_NEURO_TEST_PASS
make sanitize # AddressSanitizer/UBSan builds and runs (Linux)
make clean
```

The build is strict C11: ``-std=c11 -O2 -fPIC -Wall -Wextra -Wpedantic -Werror -fno-fast-math -ffp-contract=off`` plus reproducibility prefix maps.

The library depends only on libc.

9.2 Cross-compile matrix (zig cc)

All six targets build from any host with zig 0.13.x used as zig cc

(pip install ziglang==0.13.0). From engine/c/neuroroute/:

```
Z="zig cc"

F="-O2 -std=c11 -fPIC -fno-fast-math -ffp-contract=off -ffile-prefix-map=$PWD=. -s"

$Z -target x86_64-linux-gnu $F -shared -Wl,--build-id=none -o libneuroroute.linux-x86_64.so
neuroroute.c nr_neuro.c

$Z -target aarch64-linux-gnu $F -shared -Wl,--build-id=none -o libneuroroute.linux-aarch64.so
neuroroute.c nr_neuro.c

$Z -target x86_64-macos-none $F -dynamiclib -o libneuroroute.macos-x86_64.dylib neuroroute.c
nr_neuro.c

$Z -target aarch64-macos-none $F -dynamiclib -o libneuroroute.macos-arm64.dylib neuroroute.c
nr_neuro.c

$Z -target aarch64-linux-musl $F -shared -o libneuroroute.android-arm64.so neuroroute.c
nr_neuro.c

$Z -target x86_64-linux-musl $F -shared -o libneuroroute.android-x86_64.so neuroroute.c
nr_neuro.c
```

Android targets musl and statically links it; the resulting .so declares only libc.so (present on every Android device) and references just malloc/calloc/free/memcpy/memset, so it is dlopen-safe via ctypes, JNI, the NDK, or embedded-Python environments. On macOS, a universal2 library can be assembled on a Mac with

```
lipo -create libneuroroute.macos-x86_64.dylib libneuroroute.macos-arm64.dylib -output
libneuroroute.dylib.
```

9.3 The Owner Engine console

python3 -m owner is the owner build console. Every subcommand accepts

--json and exits 0 on success, 1 on error.

Command	Purpose	Key flags
`status`	Toolchain detection (zig cc, host cc) and health of built engines	`--dir`
`build`	Compile engine targets from C source	`--target NAME`, `--out DIR`
`verify`	Verify built engines (magic bytes, sha256, size)	`--dir`
`manifest`	Write a deterministic release manifest	`--version V` (required), `--dir`
`package`	Build all six targets, verify, write checksums + manifest into `dist/owner-engines-<version>/`	`--version V` (required)

Example release engine pass:

```
python3 -m owner status
python3 -m owner build
python3 -m owner verify
python3 -m owner package --version 1.7.6
```

Without zig, build compiles only the host target with the host cc. The desktop Tools → Owner: build engines... button runs status then build and reports Owner Engine: N/6 engine targets built.

9.4 Bundle scripts and release checks

```
python3 scripts/build_linux_bundle.py    # dist/Dexr-1.7.6-linux-x86_64/
python3 scripts/build_macos_app.py       # dist/Dexr.app
python3 scripts/run_all_checks.py         # full 17-gate release matrix
python3 main.py self-test                 # same matrix via the CLI
python3 install.py --build-macos-app     # validate + build the app in one step
```

9.5 Release checklist

46. Bump `VERSION`; review `CHANGELOG.md`.
47. `python3 main.py self-test` → `ALL DEXR CHECKS PASSED`.
48. `python3 -m owner package --version <v>` → builds ok, verified, checksums, manifest.
49. `python3 scripts/build\_linux\_bundle.py` and
 `python3 scripts/build\_macos\_app.py; runtime-smoke the Linux bundle;`
 structurally validate the .app.
50. Record results in `TEST\_REPORT.md` and `RELEASE\_VALIDATION.md`; note any
 platform not executed on physical hardware.
51. Ship the source tree, the platform bundles, or the AIO layout
 (source/, builds/<six targets>/ + SHA256SUMS.txt, docs/).

9.6 Reading TEST_REPORT.md

TEST_REPORT.md is the release's evidence file: validation date, test current suite counts, the full matrix verdict, the bit-exactness proof numbers, reproducible-build verification per target, GUI/CLI smoke results, and the IP/trademark audit verdict (SHIP). Historical sections are preserved per release. A release is only as strong as its latest entry — re-run the matrix after any change and update the report before shipping.

10. Platform notes

10.1 Linux

- The x86_64 bundle is a complete source tree plus a prebuilt libneuroroute.so and the Dextr launcher (chmod +x Dextr; ./Dextr).
- `Dextr.desktop` is included for desktop-environment integration.
- Requires Python 3.10+ with Tk (install your distribution's `python3-tk` equivalent). `make -C engine/c/neuroroute clean` all rebuilds the engine

in place for any other Linux architecture.

10.2 macOS

- ``Dexr.app`` contains the complete source tree. On first launch it tries to build the NeuroRoute `.dylib` with the Mac's compiler, then starts with `python3`. Without Xcode Command Line Tools, the bit-exact Python router is used.
- The launcher auto-selects the architecture-specific prebuilt (`libneuroroute.arm64.dylib` on Apple silicon, `...x86_64.dylib` on Intel) when present, then the plain `libneuroroute.dylib`.
- Requires macOS 12+, Python 3.10+ with Tk. The app is unsigned and unnotarized: Control-click → Open on first launch, or use `Dexr.command` from the source package. State is stored outside the bundle in `~/Library/Application Support/Dexr`.

10.3 Android

Dexr's engine ships as Android `.so` builds (`arm64` and `x86_64`). Usage paths:

- ****Python on Android** (terminal emulator environments or embedded-Python app frameworks):** drop the matching `.so` next to `engine/c/neuroroute/libneuroroute.so`; Dexr loads it automatically via `ctypes`.
- ****JNI / NDK:**** the library is ``dlopen`-safe` (only ``libc.so`` symbols).
- ****CLI mode:**** ``python3 main.py run/doctor/...`` works in Android Python environments; the desktop GUI requires a Tk-capable environment.
- ****Fallback:**** the bit-exact Python router produces identical routing scores when no native library can load, so behavior never changes.

10.4 Windows

Windows remains a source-execution platform for the desktop, deterministic engine, local-model client, file tools, memory, and reversible self-build. No verified Windows restricted-process backend is implemented in 1.7.6; therefore validation and agent command execution fail closed instead of running unsandboxed. The bit-exact Python router remains the routing fallback when no native Windows library is present.

11. Security & privacy

- ****No telemetry.**** No analytics, crash reporting, update checks, or background outbound service connections are implemented.
- ****Hardened local-model boundary.**** Model traffic is canonical numeric loopback only, proxy-free, redirect-free, credential-free, query/fragment-free, method-restricted, and request/response-size-bounded.
- ****Restricted command execution.**** Validation and agent commands run only through a reported OS backend. Linux prefers Bubblewrap; otherwise it may use user/network/PID/IPC namespaces. macOS uses Seatbelt when available. Unsupported platforms fail closed.
- ****Filesystem boundary is explicit.**** Bubblewrap and Seatbelt provide write isolation to the workspace and scratch area. The Linux namespace fallback isolates network and processes but does not add filesystem write isolation beyond the invoking account permissions; Doctor reports this difference.
- ****Environment isolation.**** HOME, TMP, and XDG state are redirected; secret-like names and proxy variables are scrubbed; user-site Python imports and interactive package prompts are disabled.
- ****Resource controls.**** Command duration, process group, stdout/stderr, argv, address space, CPU, file size, open files, and process count are bounded where supported.
- ****Workspace confinement.**** File tools resolve inside the selected workspace, reject traversal, and do not follow the desktop file-tree symlinks.
- ****Approval and recoverability.**** Writes and command actions are approval-gated. Editor saves confirm first, create timestamped backups, and use atomic replacement.
- ****Validated replay.**** Cache reuse requires the same normalized prompt, exact content fingerprint, and a prior successful validation record.
- ****Self-build isolation.**** Generated modules are syntax-checked, content-addressed, versioned, statically verified before import, promoted atomically, and stored outside the installed source.
- ****Quine integrity.**** Production quine commands reconstruct and statically inspect the expected kernel before execution, verify exact bytes and launcher modes, and reject payload, order, length, digest, root, trailing, truncation, and post-seal tampering.
- ****Local deletion boundary.**** Removing the configured state directory deletes Dexr-held threads, caches, journals, policies, and generated modules. Project files are separate.

No software can honestly be described as 100% secure. These guarantees are limited to the controls, backends, and test environments recorded in the release evidence.

12. Licensing

Dexr 1.7.6 is proprietary commercial software owned by Alex Moraru and governed by the DXR Proprietary Software License (LICENSE).

Summary (the license text governs):

- A paid copy grants only the limited installation and use rights stated in LICENSE and the applicable purchase terms. Copying, redistribution, sublicensing, resale, publication, source disclosure, modification, reverse engineering, and derivative works are prohibited unless Alex Moraru gives prior written permission or mandatory law expressly permits the activity.
- Third-party components are listed in `**`THIRD_PARTY_NOTICES.md`**` and remain under their own licenses; their required notices must be retained.
- `**AI models are licensed separately.**` Dextr includes no model weights. Any model you download is obtained by you from its official source and is governed solely by that model's own license and terms.
- The software is provided "as is" to the maximum extent permitted by law; see LICENSE.

13. Troubleshooting

Symptom	Likely cause	Fix
GUI does not open	Tk missing	<code>`python3 -c "import tkinter; print(tkinter.TkVersion)"`</code> ; install your platform's Tk package (e.g. <code>`python3-tk`</code>).
macOS blocks the app	Unsigned bundle	Control-click → Open, or run <code>`Dextr.command`</code> from the source package.
<code>`doctor`</code> exits 1	<code>`native_router: false`</code>	Native engine not built for this platform <code>`make -C engine/c/neuroroute clean all`</code> ; routing is still bit-exact via the Python fallback.
"No local model runtime was found"	Runtime not installed or not on PATH	Install a local model runtime; or pass <code>`--runtime /path/to/binary`</code> to <code>`scripts/setup_local_model.py`</code> .
<code>`Local model: unavailable`</code>	Runtime not serving, or tag not pulled	Start the runtime, pull the tag, then <code>**Tools → Check local model**</code> ; verify <code>`DEXR_LOCAL_MODEL`</code> matches the pulled tag.
Endpoint URL rejected	Remote host, credentials, query/fragment, invalid port, or unsafe path	Use a local runtime on numeric loopback or localhost; redirects and proxies are intentionally unsupported.
Command or validation blocked	No verified restricted-process backend	Run Doctor and inspect <code>command_sandbox</code> . Install Bubblewrap on Linux when appropriate; on unsupported platforms DXR 1.7.6 fails closed.
Download fails	Bad tag, runtime offline, or source issue	Re-check the tag your runtime provides; run <code>`python3 scripts/setup_local_model.py --start --pull --model <tag>`</code> to see the raw error; check the runtime log under <code>`<state>/runtime/local-runtime.log`</code> .

| "Local model setup needs attention" | Pull returned non-zero | Read the output in the Activity panel; the exit code is the runtime's pull exit code. |

| Validation fails repeatedly | Real failures, or repair budget exhausted | Read the failure text in Activity; fix or approve targeted writes; raise `--max-repair-attempts` (max 4). Dextr never marks a failing task passed. |

| "this Dextr thread already has an active turn" | Concurrent submit on one thread | Wait for the turn, press `**Stop**`/`Escape`, or start a new thread. |

| Approval dialog seems stuck | 600 s timeout pending | Answer the dialog; after 600 s the request auto-cancels. |

| Old answer reappears | Validated replay hit | Workspace and prompt exactly match a previous success. Uncheck `**Replay validated cache**` or use `--no-cache`. |

| Editor refuses a file | > 2 MB or binary | The editor limit is 2 MB UTF-8 text; use an external editor for larger/binary files. |

| ``policy`` looks wrong / routes drifting | Accumulated bias | ``python3 main.py policy reset --workspace <path>``. |

| Self-build module rejected | Syntax/compile/check failure | The error message names the failure; fix the source and resubmit – rejected candidates are journaled, never promoted. |

| State growing large | Journals, caches, versions | Delete the Dextr state directory (\$2.5). Projects are untouched. |

14. FAQ

Does Dextr need the internet?

No. Deterministic mode is fully offline. The only possible network use is a loopback connection to a local model runtime on your own machine, and model downloads you explicitly start.

Does Dextr include an AI model?

No. Weights are never bundled. You choose one of four generic engine tiers (7B/14B/20B/32B) as a sizing guide and supply your runtime's model tag.

What can I use without a model?

Everything except model-backed agents: the desktop, deterministic generation, routing, validation, repair bookkeeping, replay cache, memory, self-build, CLI, and all deterministic tests.

Which local model runtime do I need?

Any runtime with a chat-completions-compatible loopback API. Dexr detects a runtime CLI on PATH (or accepts --runtime PATH) and talks to the endpoint you configure (default `http://127.0.0.1:11434/v1`).

Can I point Dexr at a hosted API?

No. Non-loopback hosts are rejected in code; there is no override.

Is my code sent anywhere?

No. There is no telemetry and no remote endpoint. Files are read only inside the workspace you open.

What exactly runs when I press Send?

Route scoring (NeuroRoute) → validated-replay check → backend start → deterministic generation or the recursive agent mesh → optional validation → bounded repair on failure → experience/policy update. Every step is journaled locally.

Can Dexr modify files without asking?

No. Writes and commands are approval-gated per action; the editor asks before saving and keeps backups in `.dexr-backups`.

What happens when validation keeps failing?

Repair runs up to your budget (default 2, max 4 attempts). If it still fails, the task is marked failed with the real failure output preserved.

What is "validated replay" and can I turn it off?

If you repeat an identical prompt in an unchanged workspace and the previous run validated, Dexr replays that exact answer. Turn it off with the `**Replay`

validated cache** checkbox or --no-cache.

Does Dexr learn or train models?

It stores validated experiences, replay entries, tool-cache results, and small bounded routing biases (± 3.0 , decaying). It never trains or modifies model weights.

Does self-build modify the installed app?

No. Generated modules are validated, versioned, and rolled back inside the state directory only.

Are routing results the same on every machine?

The implementation is designed for that property, and native/fallback output is compared bit-for-bit. A machine or target is claimed only after the test suite runs there. See BUILD_INFO.json and VERIFICATION_REPORT.md for the currently executed target.

How do I completely reset Dexr?

Close it and delete the state directory (§2.5). Your project files are never deleted by Dexr.

Is the macOS app signed?

No. It is structurally validated but unsigned and unnotarized; use Control-click → Open.

What license applies to models I download?

Each model's own license from its authors, shown at the official download source. The proprietary DXR license covers only DXR-owned software; downloaded models retain their own licenses.

15. Glossary

Term	Meaning
Agent mesh	The bounded recursive tree of specialist agents that executes model-backed tasks.
Approval gate	The per-action permission dialog/callback required before any write or command.
Arm (Octopus)	One of twelve registered capabilities (local-model, neuroroute, planner, repository, implementer, tester, critic, repair, cache, history, builder, tools) reported in the GUI and doctor.
Backend	The active execution path: <code>`local-model`</code> , <code>`deterministic`</code> , or <code>`validated-replay`</code> .
Bit-exact fallback	The pure-Python router that reproduces the C router's arithmetic with zero bit differences.
Bounded repair	Re-running a task with verified failure text, capped at 0-4 attempts.
Deterministic mode	Operation using NeuroRoute routing and templates only – no model, no network.
Doctor	The readiness/boundary report (<code>`main.py doctor`</code> , desktop Doctor button).
Engine tier	A generic model size class (Compact 7B / Balanced 14B / Standard 20B / Performance 32B) used as hardware guidance.
Event journal	The per-workspace redacted JSONL event log (<code>`events.jsonl`</code>).
Loopback rule	The hard restriction of model traffic to <code>`127.0.0.1`</code> / <code>`localhost`</code> / <code>`:::1`</code> .
Model tag	The identifier your local model runtime assigns to a model (e.g. set via <code>`DEXR_LOCAL_MODEL`</code>).
NeuroRoute	Dexr's deterministic C11 routing engine (24-32-8 MLP plus the extended <code>`nr_*`</code> graph router and spatial pool).
Owner Engine	The owner build console (<code>`python3 -m owner`</code>) for compiling, verifying, and packaging the six native engine targets.
Padé activation	The Padé (7/6) tanh approximant used as the router activation; pure IEEE arithmetic, no libm.
Policy (adaptive)	The per-workspace bounded bias layer (± 3.0 , decaying) applied to routing scores.
Repair role	The agent role that plans corrections from verified validation failures only.
Self-build registry	The reversible, manifest-driven store for generated runtime modules (validate → version → promote → rollback).
Spatial pool	The 256-bit-coordinate, Hamming-matched sparse memory pool in the extended NeuroRoute API.
Spike counter	The atomic per-edge route-use counter in the <code>`nr_*`</code> graph router.
State directory	The user-level directory holding Dexr's database, journals, caches, policies, and generated modules.
Strategy	One of eight routing outcomes: function, class, test, refactor, import, print, explain, fallback.
Thread	A persistent task conversation stored per workspace.

| Validated replay | Exact answer reuse requiring identical prompt, identical workspace fingerprint, and prior validation success. |

| Workspace | The single project directory Dextr is confined to. |

Dextr Owner & User Manual — v1.7.6

Quick Start

52. Extract `DXR-1.7.6-Quine-AIO`.
53. Confirm Python 3.10 or newer: `python3 --version`.
54. Run `python3 main.py doctor --workspace /path/to/project`.
55. Start the desktop with `python3 main.py desktop --workspace /path/to/project`.
56. For a non-GUI deterministic task, run `python3 main.py run --workspace /path/to/project --prompt "Explain this project" --backend deterministic`.
57. Before modifying or releasing DXR, run `python3 main.py self-test`.

DXR stores runtime state outside the workspace. Set `DEXR_STATE_DIR` to select a custom state location. The 1.7.6 verifier always replaces this variable temporarily during self-test, preventing old replay/cache state from contaminating release validation.

Usage Reference

Requirements

Minimum:

- Python 3.10 or newer;
- Tk support for the desktop interface;
- a writable user state directory.

Recommended:

- Git for repository fingerprints and diff display;
- a C compiler and Make for the native NeuroRoute engine;
- Any local model runtime with a chat-completions-compatible loopback API, for model-backed agents.

Dexr works in deterministic mode without a model runtime.

Linux x86_64 package

```
unzip Dexr-1.7.6-linux-x86_64.zip
cd Dexr-1.7.6-linux-x86_64
chmod +x Dexr
./Dexr
```

To open a specific project:

```
DEXR_WORKSPACE="$HOME/Projects/example" ./Dexr
```

macOS package

58. Unzip `Dexr-1.7.6-macOS.zip`.
59. Move `Dexr.app` to Applications if desired.

60. Open the app.
61. Choose a project folder in the top bar.

The app is not signed or notarized. macOS may require Control-click → Open for the first launch. It uses the python3 available on the Mac. Install Python with Tk and Xcode Command Line Tools when the native router or GUI cannot start.

The included Dextr.command can also be used from the source package.

First desktop workflow

62. Choose a workspace.
63. Select `auto`, `local-model`, or `deterministic`.
64. Use ****Check model**** to inspect the local endpoint.
65. Create a thread.
66. Enter a coding request.
67. Review any write approval dialog.
68. Inspect Activity, Changes, Memory, and Tools.
69. Run validation before accepting generated changes.

The Editor tab opens text files up to 2 MB. Saving asks for confirmation and writes a backup under .dextr-backups.

Local model setup

With a local model runtime already installed:

```
python3 scripts/setup_local_model.py --start --pull --model dextr-local
```

Use another local model:

```
python3 scripts/setup_local_model.py --start --pull --model your-model-tag
export DEXR_LOCAL_MODEL=your-model-tag
```

Use another loopback-compatible endpoint:

```
export DEXR_LOCAL_MODEL_URL=http://127.0.0.1:1234/v1
```

A non-loopback hostname is rejected by design.

Useful optional settings:

```
export DEXR_LOCAL_MODEL_TIMEOUT=180
export DEXR_LOCAL_MODEL_REASONING=medium
export DEXR_LOCAL_MODEL_MAX_TOKENS=4096
export DEXR_STATE_DIR="$HOME/.dextr-state"
```

CLI

Show help:

```
python3 main.py --help
```

Open desktop:

```
python3 main.py desktop --workspace /path/to/project --start-backend
```

Inspect readiness and boundaries:

```
python3 main.py doctor --workspace /path/to/project
```

Run one task:

```
python3 main.py run \
  --workspace /path/to/project \
  --prompt "Implement the parser and add tests" \
  --backend auto \
  --recursive-depth 3 \
```

```
--max-repair-attempts 2
```

Deterministic-only task:

```
python3 main.py run \  
  --workspace /path/to/project \  
  --prompt "Create a small Python function" \  
  --backend deterministic \  
  --no-validation
```

Direct deterministic generator:

```
python3 main.py prompt "def fibonacci(n):"
```

Compatibility recursive deterministic agent:

```
python3 main.py agent --prompt "def add(a, b): then class Calculator:" --max-depth 2
```

Show or reset adaptive route policy:

```
python3 main.py policy show --workspace /path/to/project  
python3 main.py policy reset --workspace /path/to/project
```

Run the complete release checks:

```
python3 main.py self-test
```

Validation behavior

By default, Dextr discovers:

- Python syntax checks when Python files exist;

- Python `unittest` discovery when a `tests/` directory contains `test\_\*.py`.

Custom direct commands can be supplied through the Python API. Pipelines, redirects, &&, and other shell operators are rejected. Validation output and exit status are shown in the Activity panel.

Reversible self-build

The Tools panel can create a named generated module. Dextr validates and versions it before activation. Existing active versions remain available for rollback. Generated modules live in the user state directory, not in the installed app.

Data locations

The local database contains project paths, threads, messages, events, and validated experiences. Each workspace also has isolated policy, cache, local-model journal, and self-build data.

Delete the Dextr state directory to remove this local history. Project files are not deleted by that action.

Troubleshooting

GUI does not open

Confirm Tk is available:

```
python3 -c "import tkinter; print(tkinter.TkVersion)"
```

Local model unavailable

```
your-runtime list
```

```
your-runtime serve
```

```
python3 scripts/setup_local_model.py --json
```

Native router unavailable


```
make -C engine/c/neuroroute clean all  
python3 main.py doctor --workspace .
```

Dexr uses its deterministic Python router if the native library cannot load.

macOS blocks the app

The generated app is unsigned. Control-click the app and choose Open, or run Dexr.command from the source package. Signing and notarization are not part of this release.

Payload-quine commands

```
python3 main.py quine inspect  
mkdir /tmp/dexr-gen1  
python3 main.py quine reproduce --output /tmp/dexr-gen1  
python3 main.py quine verify --report FIXED_POINT_PROOF.json --work-dir /tmp/dexr-proof --keep-work
```

Use `--state-dir` immediately after quine to select the content-addressed promotion registry. Promotion requires the successful proof file and a candidate whose root matches all three proven generation roots.

Part II — Architecture

1. Application boundary

Dexr is a standalone, self-contained process tree with no external application integrations. The only optional model connection is an HTTP client restricted to a loopback address selected by the user.

The high-level path is:

Desktop or CLI

|

DexrOrchestrator

|

- +-- Local recursive agent mesh (optional loopback model)
- +-- Deterministic NeuroRoute/template engine
- +-- Octopus arm registry
- +-- Workspace tools and approvals
- +-- Validator and bounded repair
- +-- Session/experience store and event journal
- +-- Reversible self-build registry

2. Desktop shell

app/desktop.py provides a Tk desktop workspace with:

- project picker and file tree;
- persistent local threads and transcript;
- Agent and Editor views;
- explicit file save with confirmation and backup;
- Activity, Changes, Memory, and Tools tabs;
- validation, Git diff, model health, and dynamic-module controls;
- approval dialogs for runtime mutations;
- turn cancellation and background event delivery.

Desktop state defaults to:

- macOS: `~/Library/Application Support/Dexr`
- Linux: `\${XDG\_STATE\_HOME:-~/.local/state}/Dexr`
- Windows-compatible source execution: `%LOCALAPPDATA%/Dexr`

DEXR_STATE_DIR overrides the location.

3. Orchestrator

app/orchestrator.py owns the lifecycle of a selected workspace. It:

70. creates isolated workspace state;
71. fingerprints the repository or filesystem state;
72. selects `local-model` or `deterministic` execution;
73. creates and persists threads/messages;
74. emits redacted events;
75. applies exact validated replay only when the workspace fingerprint matches;
76. delegates to recursive agents or the deterministic core;
77. runs validation;
78. performs bounded repair when enabled;
79. stores successful experience and adjusts bounded policy weights.

Backend choices are limited to auto, local-model, and deterministic.

4. Local model boundary

engine/llm/local_runtime.py is the only model transport. It canonicalizes localhost to numeric loopback, disables environment proxies, refuses redirects and credentials, rejects query/fragment/path confusion, and bounds HTTP bodies. The local model runtime and model weights are user-supplied and separately licensed.

5. Recursive agent mesh

agents/recursive_runtime.py implements a bounded task graph. Roles include:

- planner;

- repository analyst;
- implementer;
- test designer;
- critic;
- repair agent;
- synthesizer.

Limits constrain depth, total nodes, tool calls, and batch width. Independent read-only work can run in parallel. Write operations use the approval callback. The agent does not receive unrestricted filesystem or shell access.

6. Octopus system

engine/octopus_runtime.py maintains the available-arm registry surfaced in the GUI and doctor report. The current arms are:

```
80. local-model;
81. neuroroute;
82. planner;
83. repository;
84. implementer;
85. tester;
86. critic;
87. repair;
88. cache;
89. history;
90. builder;
91. tools.
```

The registry reports capability and health; it is not an uncontrolled process spawner.

7. Deterministic core

engine/c/neuroroute/ contains the C11 routing engine. engine/octopus.py loads the native shared library when present and falls back to a deterministic Python scoring implementation otherwise.

brain/polymorphic_core.py combines the route with deterministic templates, cache, validation metadata, and mutation records. This path runs without a local model and without network access.

8. Workspace tools

`tools/workspace_tools.py` confines file paths to the selected workspace and accepts only allowlisted bare executable names. Commands are parsed without a shell, resolved to an absolute executable, and delegated to `integration/secure_process.py`. The active OS backend, resource limits, environment isolation, output caps, and fail-closed result are part of the command result boundary.

9. Validation and repair

`integration/validation.py` discovers project checks, rejects shell grammar, and executes direct `argv` through the same restricted-process runner used by agent commands. Validation results include exact exit codes, timeouts, bounded output, and sandbox capability fields. Verified failure text alone enters the repair prompt, and the bounded repair budget cannot convert a failing check into a pass.

10. Persistence and bounded adaptation

`app/session_store.py` stores projects, threads, messages, events, and validated experiences in SQLite.

`brain/adaptive_policy.py` stores small bounded route biases. Exact replay requires both the same normalized prompt and the same workspace fingerprint. The event journal redacts common secret fields before writing JSONL.

This is application-level adaptation, not model-weight training.

11. Reversible self-build

`selfbuild/registry.py` creates isolated generated Python modules under workspace-specific state. A candidate must parse and pass its declared checks before promotion. Active versions are selected by a manifest pointer and can be rolled back.

The self-build system does not overwrite Dextr's installed source tree, executable launcher, or model runtime.

12. Packaging

- Linux x86_64 bundle: complete source plus a prebuilt native `.so`` and launcher.

- macOS bundle: source-contained `.app``; native `.dylib`` is built on the target Mac when a compiler is available.
- source release: platform-neutral Python/C source and build scripts.

No model weights, local inference runtime, Python interpreter, or third-party coding application are bundled.

Deterministic payload-quine subsystem

The real CLI path is `main.py -> quine.cli -> quine.service`. The service statically parses the generated kernel, reconstructs and compares its expected source bytes, validates the embedded DXRCAP01 capsule, checks its AST policy, and seals the current canonical tree. It executes only a private copy of those already-verified kernel bytes. The kernel reconstructs files into an explicit empty directory and never imports or promotes them.

`quine.canonical` defines portable paths, modes, exact-byte SHA-256 leaves, and the Merkle root. `quine.registry` stores successfully proven candidates in content-addressed version directories and atomically moves a pointer; it never overwrites the running source tree. Derived environment reports are excluded from the fixed-point payload root.

Canonical Capsule Design

The embedded capsule is base85 text containing the following binary structure:

```
8 bytes    magic = DXRCAP01
4 bytes    unsigned entry count, big-endian
8 bytes    unsigned body length, big-endian
N bytes    entry body
32 bytes    payload Merkle root
```

Each entry is:

```
2 bytes    path length
P bytes    canonical ASCII path
1 byte     type = 1 (regular file)
2 bytes    mode (0644 or 0755)
8 bytes    exact payload length
32 bytes    SHA-256 of exact payload bytes
S bytes    payload
```

Entries are strictly sorted by raw path bytes and duplicates are rejected. The capsule excludes the kernel itself; the kernel reproduces its own exact bytes through its literal quine structure and adds that file to the full canonical root. The capsule root protects all non-kernel payload entries. The full source root is calculated over those entries plus the reproduced kernel.

Leaf and node formulas are:

```
leaf = SHA256("DXRLEAF1" || u16(path_len) || path || u16(mode)
              || u64(size) || file_sha256)
node = SHA256("DXRNODE1" || left || right)
empty = SHA256("DXREMPY1")
```

An odd Merkle level duplicates its last node. Truncation, extension, mutation, unsupported type/mode, unsafe path, duplicate/order change, digest mismatch, or root mismatch is a hard error.

Machine Verification Design

Source fixed-point proof and native binary reproducibility are separate gates.

For each actually tested target, record compiler/linker versions, target, flags, environment, and two independently reproduced source trees. Build the NeuroRoute shared library twice using deterministic flags and path remapping, compare exact SHA-256, inspect binary metadata when host tools are available, and run the C unit tests plus sanitizers where supported.

No report may generalize a successful host build to an unexecuted target.

Cross-target prebuilt binaries remain distributable application payload files, but their historical provenance is not converted into a new reproducibility claim without rebuilding them in the current verification environment.

Protocol and Policy

PROTOCOL_V1 is the application-owned protocol for deterministic source reproduction. It takes precedence over ad-hoc stdout capsule ideas.

```
PROTOCOL_V1 = {  
    magic: DXRCAP01,  
    path_encoding: strict portable ASCII relative paths,  
    byte_order: big-endian,  
    ordering: ascending raw path bytes,  
    file_types: regular files only,  
    modes: 0644 or 0755,  
    hash: SHA-256 over exact stored bytes,  
    tree: DXR-MERKLE-SHA256-v1,  
    process_count: NPROC=1,  
    execution: reconstructed application is never imported or run,  
    promotion: external verified registry only,  
    reports: derived and excluded from the payload root  
}
```

The canonical file set is discovered from the real source tree at kernel build time under the explicit exclusion policy in `quine/canonical.py`; no hard-coded file count is trusted as an integrity claim.

Sandbox Guardrails

The application has two separate boundaries: workspace-confined file operations and OS-restricted process execution. Approval does not disable either boundary.

Local-model networking

- Only HTTP/HTTPS numeric loopback is used after canonicalization.
- Proxies, redirects, credentials, query strings, fragments, unsafe paths, and oversized bodies are rejected.

Restricted-process backends

- Linux Bubblewrap: read-only host exposure, writable workspace/scratch binds, new session, unshared namespaces, and no network.
- Linux namespace fallback: user/network/PID/IPC namespaces and process isolation; filesystem writes remain governed by the caller account.
- macOS Seatbelt: default deny, file reads allowed, writes limited to workspace/scratch, network denied.
- No verified backend: return code 126 and blocked execution; never silently run unrestricted.

Process guard

- Sets a new session/process group, deterministic state directories, secret/proxy scrubbing, direct executable resolution, and resource limits.
- Timeout termination targets the whole process group; stdout and stderr are captured to bounded temporary files and returned as tails.

Operator responsibility

Approved code can still consume resources within the imposed caps and read files exposed by the active backend. Operators must review the exact command and Doctor capability report.

Part III — Build and Packaging

This document describes how the native NeuroRoute library is built for every supported platform, how reproducibility is enforced, and how to assemble the release bundles.

Requirements

Task	Tool
Native Linux/macOS build	any C11 compiler (gcc/clang/cc), `make`

| Cross-compile (all targets) | [zig](https://ziglang.org) 0.13.x used as `zig cc` (`pip install ziglang==0.13.0`) |

| App runtime | Python 3.10+ with Tk |

| Tests | Python stdlib `unittest` only |

No other native dependencies exist. The library only uses libc.

Quick build (host platform)

```
cd engine/c/neuroroute

make          # produces libneuroroute.so (Linux) or .dylib (macOS)

make test     # ROUTER_TEST_PASS + NR_NEURO_TEST_PASS

make sanitize # ASan/UBSan run of both test binaries (Linux)
```

Cross-compile matrix (zig cc)

From engine/c/neuroroute/:

```
Z="zig cc"

F="-O2 -std=c11 -fPIC -fno-fast-math -ffp-contract=off -ffile-prefix-map=$PWD=. -s"

# Linux

$Z -target x86_64-linux-gnu $F -shared -Wl,--build-id=none -o libneuroroute-linux-x86_64.so
neuroroute.c nr_neuro.c

$Z -target aarch64-linux-gnu $F -shared -Wl,--build-id=none -o libneuroroute-linux-aarch64.so
neuroroute.c nr_neuro.c

# macOS (universal possible via lipo on a Mac)

$Z -target x86_64-macos-none $F -dynamiclib -o libneuroroute-macos-x86_64.dylib neuroroute.c
nr_neuro.c

$Z -target aarch64-macos-none $F -dynamiclib -o libneuroroute-macos-arm64.dylib neuroroute.c
nr_neuro.c

# Android (static musl link; only libc.so symbols are referenced,
# which Android's bionic provides – dlopen-safe via ctypes/JNI)

$Z -target aarch64-linux-musl $F -shared -o libneuroroute-android-arm64.so neuroroute.c nr_neuro.c
```

```
$Z -target x86_64-linux-musl $F -shared -o libneuroroute-android-x86_64.so neuroroute.c nr_neuro.c
```

Notes:

- **Android**: zig's pip distribution does not ship Android bionic headers, so the Android builds target musl and statically link it. The resulting .so declares only libc.so as a dependency (present on every Android device) and references just malloc/calloc/free/memcpy/memset. Load it with the NDK, Chaquopy, Termux (ctypes.CDLL), or JNI. On Android with Python (Termux/Chaquopy), drop the matching .so next to engine/c/neuroroute/libneuroroute.so and Dexr uses it automatically — the bit-exact Python fallback produces identical routing scores either way.
- **macOS**: place `libneuroroute.dylib` in `engine/c/neuroroute/`. To build a universal2 library on a Mac: `lipo -create libneuroroute-macos-x86\_64.dylib libneuroroute-macos-arm64.dylib -output libneuroroute.dylib`.

Determinism guarantees

92. **Routing scores** are platform-independent: the Padé (7/6) activation is pure IEEE double arithmetic (no libm), built with `-ffp-contract=off` and no fast-math. The pure-Python fallback is bit-exact against the C router (proved in `tests/unit/test_router_equivalence.py`).
93. **Reproducible binaries**: with the flags above, two independent builds of the same source produce byte-identical artifacts (verified for all six targets; see the checksums produced by the build).

Release bundles

```
python3 scripts/build_linux_bundle.py    # dist/Dexr-1.7.6-linux-x86_64/
python3 scripts/build_macos_app.py       # dist/Dexr.app
python3 scripts/run_all_checks.py        # full 17-gate release matrix
```

The AIO package layout (Dexr-1.7.6-AIO.zip):

```
Dexr-1.7.6-AIO/
├─ source/           # complete enhanced source tree (this tree)
├─ builds/
│   ├─ linux-x86_64/ # libneuroroute.so + bundle
│   ├─ linux-aarch64/ # libneuroroute.so
│   ├─ macos-x86_64/ # libneuroroute.dylib
│   ├─ macos-arm64/  # libneuroroute.dylib
│   ├─ android-arm64/ # libneuroroute.so
│   └─ android-x86_64/ # libneuroroute.so
│   └─ SHA256SUMS.txt
└─ docs/             # README, USAGE, ARCHITECTURE, BUILD, CHANGELOG
```

Rebuilding the payload-quine kernel

Make all canonical source changes first, then run:

```
python3 scripts/build_quine_kernel.py --root .
python3 main.py quine inspect
python3 main.py quine verify --report FIXED_POINT_PROOF.json
```

Any later canonical file change intentionally invalidates the embedded capsule;
rebuild the kernel and rerun every earlier gate. The generated kernel is a
release input and must not be manually edited.

Maintenance and Repair Runbook

Golden rule

Never edit generated quine metadata or the embedded kernel manually. Change canonical source, run the official builder, then execute the complete verifier. Keep the last verified archive until the new release passes every gate.

Routine maintenance

94. Work in a clean copy or version-control branch.
95. Make one bounded change.
96. Run focused tests for the changed module.
97. Run ``python3 main.py self-test``.
98. Rebuild quine artifacts with the packaged builder.
99. Run ``python3 scripts/verify_quine_release.py --output <fresh-directory>``.
100. Compare the generated canonical root and inspect all changed manifest entries.
101. Package only after all gates pass.

Repair decision tree

- Syntax/import failure: restore the last known-good file, reproduce the failure with the smallest test, correct the source, rerun unit tests, then full self-test.
- Native build failure: verify compiler availability, clean temporary objects, rebuild with the official Makefile, run sanitizer tests, then verify packaged binaries against rebuilt binaries.
- Fixed-point mismatch: do not hand-edit hashes. Inspect missing/extra/changed lists, remove generated contamination, rebuild the kernel and canonical manifest, then reproduce `gen0→gen1→gen2`.
- Replay/cache-dependent test: set a fresh ``DEXR_STATE_DIR``; DXR 1.7.6 self-test does this automatically. If a focused test depends on state, use a temporary directory in the test itself.
- GUI failure: run ``doctor``, verify Tk availability, then use deterministic CLI mode while repairing the GUI dependency.
- Corrupted runtime state: close DXR, back up the state root, move it aside, run ``doctor``, and reintroduce only required data. Project files remain separate.

Rollback

Restore the last verified archive, verify its SHA-256, extract to a new directory, run `python3 main.py self-test`, and only then replace the failed deployment.

GitHub Release Checklist

- [] Version values agree across `VERSION`, documentation, package names, and changelog.
- [] Proprietary `LICENSE` naming Alex Moraru is present; third-party notices remain intact.
- [] UTF-8 scan reports no replacement characters or mojibake markers.
- [] No credentials, private keys, tokens, local paths, caches, databases, or editor files are packaged.
- [] `python3 main.py self-test` exits 0 from a dirty caller environment.
- [] Full quine verification exits 0 in a fresh output directory.
- [] Test count, canonical root, kernel hash, and binary hashes are recorded.
- [] gen0 equals gen1 and gen1 equals gen2.
- [] Tamper negative controls fail as expected.
- [] Native rebuilt binary matches the packaged target on the verified host.
- [] Archive extracts successfully and its checksum file verifies.
- [] Documentation audit contains no duplicate paragraphs or broken diagram references.
- [] Known limitations are accurate and visible.
- [] Release notes clearly distinguish verified platforms from generated-but-unverified targets.

Part IV — DXR 1.7.6 Security and State-Isolation Update

1.7.6 scope

DXR 1.7.6 preserves the desktop, recursive agent, NeuroRoute, quine, self-build, packaging, and owner workflows while hardening two publication-critical boundaries: local-model networking and approved command execution.

Local-model correction

The endpoint is canonicalized to numeric loopback, proxy environment routing is disabled, redirects are rejected, credentials/query/fragment/path confusion is blocked, and request/response sizes are capped. Regression tests cover each condition.

Restricted-process correction

Validation and agent commands now use one fail-closed runner with isolated state directories, broad secret and proxy scrubbing, executable resolution, resource limits, process-group timeout termination, and explicit backend capability reporting.

State-isolated release testing

python3 main.py self-test creates a fresh temporary DEXR_STATE_DIR for the complete matrix and restores the caller environment. The matrix also invokes plain unittest discovery from its own fresh state directory, so persistent validated replay cannot contaminate release acceptance.

Archive and fixed-point verification

The release process audits fresh ZIP extraction, executable launcher modes, canonical bytes, gen0/gen1/gen2 equality, tamper rejection, native runtime behavior, and source-isolation invariants.

Boundary statement

Linux namespace fallback does not add filesystem write isolation beyond the invoking account. macOS Seatbelt was not executed on the Linux verification host. Unsupported command platforms fail closed. These limits are disclosed rather than upgraded into unexecuted claims.

Release Validation

The supported release gate is python3 main.py self-test or scripts/run_all_checks.py from a clean extracted tree and fresh state. The 1.7.6 source defines 17 gates and executes the Python suite twice per matrix: once in-process and once through plain unittest discovery. Every figure below was executed on the qualification host and is reproduced in the evidence shipped with the release.

- Official release verifier: PASSED overall, 7 of 7 gates — static and canonical structure, full test suite, CLI wiring, runtime behaviour, fixed point, tamper controls and native engine.

- 17-gate release matrix: 17/17 PASS, terminating in ALL DEXR CHECKS PASSED, and completing on macOS as well as Linux.
- Python suite: 132 passed, 0 failed, 0 skipped — executed four separate ways (official loader, plain discovery from the source root, packaging fresh-extraction audit, and an independent fresh-extraction re-run).
- Suite against a populated state directory: 132 passed — identical to a freshly imaged machine.
- Python syntax: 109 of 109 source files compile.
- Two-generation fixed point: `gen0 == gen1 == gen2`; zero missing, extra or changed canonical entries.
- Compile-then-verify: after building the native engine inside a fresh extraction, the checksum manifest verifies in full and the fixed point is still proven.
- Tamper controls: 11 passed, 0 failed, with independent injections rejected and the altered file named exactly.
- Native engine: strict C11 build under `-Werror`; router and neural unit tests pass; AddressSanitizer and UndefinedBehaviorSanitizer clean; two independent clean rebuilds byte-identical.
- Instrumented desktop and template workflow: 25 of 25 checks, exercising every main-window control through its real handler and taking a generated file through create, read, list, compile, execute, edit, atomic save with backup and reload.
- Workspace precedence: all three launch modes verified live — contextless launch, launch from inside a project, and launch from a plain home directory.
- Packaging: archive integrity clean, independent archive audit clean, and a fresh extraction ran the complete suite.
- Standalone binary and application metadata verified on the packaged builds.

Evidence precedence: use live `main.py` quine inspect output and final verification logs for the exact shipped tree. Regenerate root-level derived reports after any canonical source change; derived verification files are intentionally excluded from the payload Merkle root.

Known Limitations

- No software can honestly be described as 100% secure. DXR 1.7.6 is a hardened, fail-closed release limited to recorded controls and environments.
- Source fixed-point verification covers canonical bytes and modes. Native binary reproducibility is claimed only for targets actually built twice.
- The local-model client is proxy-free, redirect-free, response-size-bounded, and numeric-loopback-only after canonicalization. Remote endpoints remain unsupported.
- Linux prefers Bubblewrap. Without it, namespace isolation blocks network and separates processes, but filesystem writes remain limited only by the invoking account permissions.
- macOS Seatbelt support requires `/usr/bin/sandbox-exec` and was not executed on the Linux verification host.
- Platforms without a verified command sandbox fail closed for validation and agent command execution. Workspace file reads/writes remain separately confined and approval-gated.
- Approved code can consume CPU, memory, and disk within caps and can read data exposed by the active backend.
- Canonical paths use a portable ASCII subset; Unicode paths require a future versioned canonicalization rule.
- Derived verification reports are excluded from the payload Merkle root; they describe the environment or are deterministically regenerated.

- Direct execution of the raw quine kernel bypasses the production static pre-execution gate. Use `python3 main.py quine ...`.

Third-Party Notices

Dexr is proprietary commercial software owned by Alex Moraru. It incorporates the third-party components listed below, each licensed under its own license. These notices are provided to comply with the attribution requirements of those licenses and must accompany all copies of the software.

NeuroRoute (graph routing library and developer tools)

Portions of `engine/c/neuroroute/nr_neuro.c` and `nr_neuro.h` are derived from the NeuroRoute project.

- License: MIT License
- Copyright (c) 2026 fireside25a-beep

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR

IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

musl libc (statically linked into Android builds of libneuroroute)

- License: MIT License
- Copyright (c) 2005–2020 Rich Felker, et al.

(Full MIT license text as reproduced above applies.)

compiler-rt (runtime support library linked into cc-built binaries)

- License: Apache License 2.0 with LLVM Exceptions
- License text: compiler-rt LICENSE.TXT (Apache 2.0 with runtime exception)

zig cc (build toolchain only — not distributed with the app)

- License: MIT License
- Copyright (c) 2015–2024 Zig Software Foundation

Python (runtime required by the app — not distributed with the app)

- License: Python Software Foundation License 2.0
- <https://www.python.org/psf/license/>

Release Notes

DXR 1.7.6

- Launching Dextr from a terminal inside a project opens that project. Launches with no directory context reopen your last project, or the dedicated DXR-Projects folder on a first run.
- Projects are kept in a dedicated DXR-Projects folder in your home directory, created automatically, so the project tree stays clear of operating-system managed directories.
- Compiling the native engine writes every build product into a separate build directory, so a local compile never disturbs the sealed release tree and verification still succeeds afterwards.
- The complete seventeen-gate release matrix runs to completion on macOS as well as Linux, with the sanitizer gate adapting to the platform runtime and checking the sanitized router's explicit pass marker.
- Release packaging stamps application version metadata automatically.
- Local-model networking is restricted to a canonical numeric loopback address, with proxy routing, redirects and hostname rebinding refused.
- Command execution runs through a restricted-process runner with isolated state, secret and proxy scrubbing, and platform sandbox backends; unsupported platforms fail closed.
- The embedded capsule, canonical metadata, release manifests, checksums and verification evidence are regenerated for every release.

Appendix A — Verification Evidence

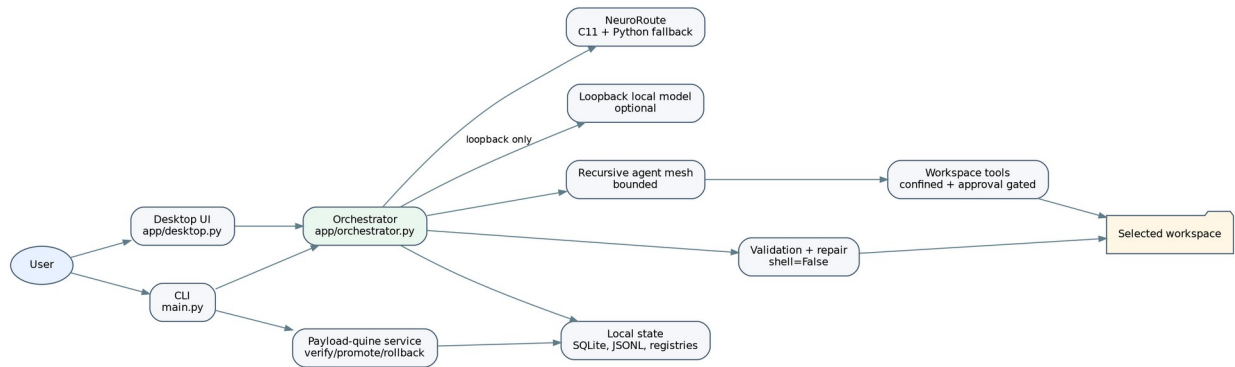
The 1.7.6 verification evidence records the source, runtime, fixed-point, tamper, native and sandbox gates. The table above summarises the principal gates.

Gate	Result	Evidence
Static and canonical	PASS	canonical structure, manifests and checksums verified against the shipped tree
Test suite	PASS	132 passed, 0 failed, 0 skipped; executed four ways
CLI wiring	PASS	production command paths exercised, including the quine entry point
Runtime	PASS	doctor, deterministic task execution, policy and workspace behaviour
Fixed point	PASS	gen0 == gen1 == gen2 across two reproduction generations
Tamper	PASS	11 passed, 0 failed
Native engine	PASS	strict C11 build, unit tests, sanitizers clean, rebuilds byte-identical
Desktop workflow	PASS	25 of 25 instrumented GUI and template checks

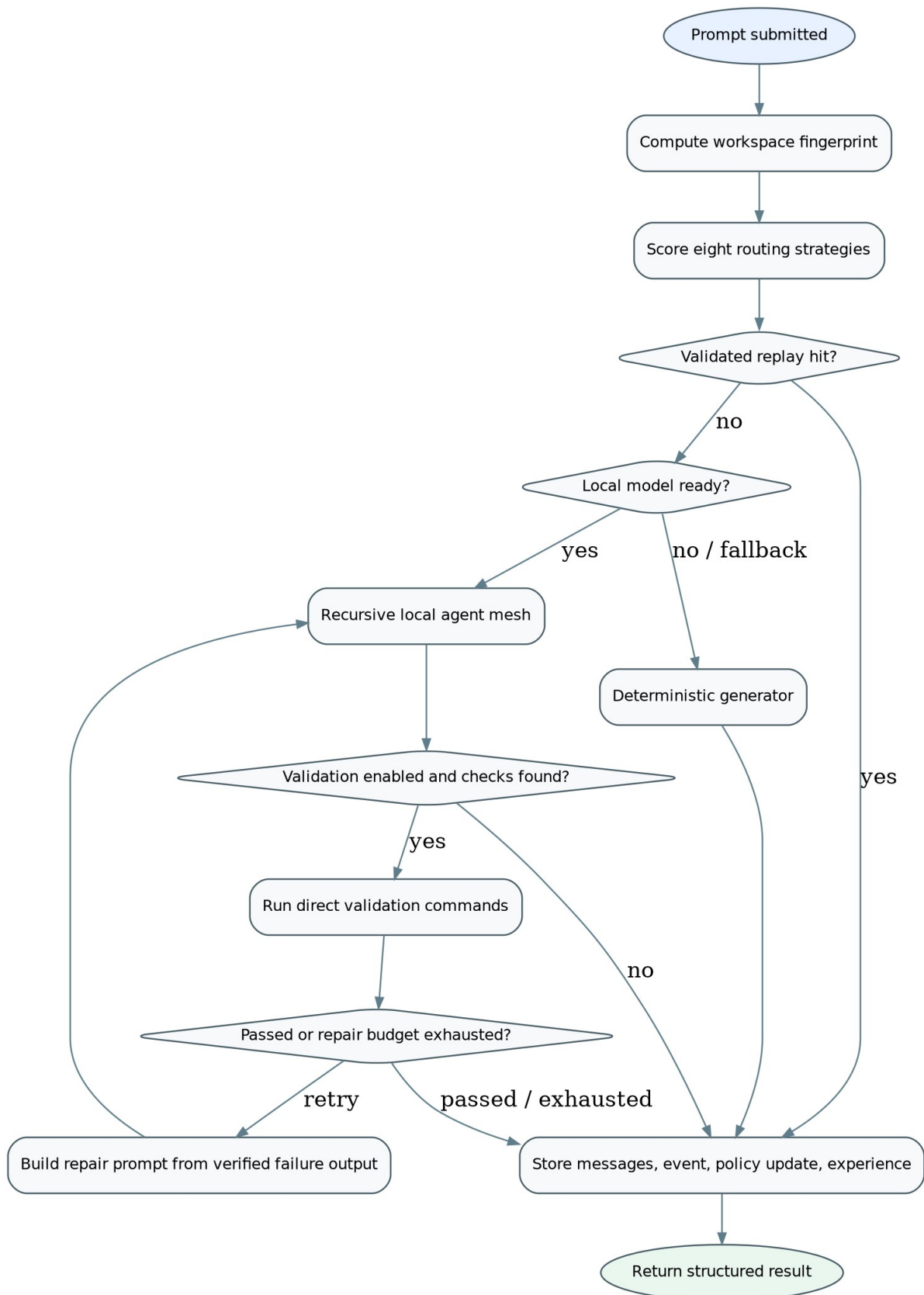
Complete evidence is supplied with the release, including the verification summary, the fixed-point proof, the per-gate logs, the canonical manifest and the checksum manifest.

Appendix B — Schematics

01 System Architecture



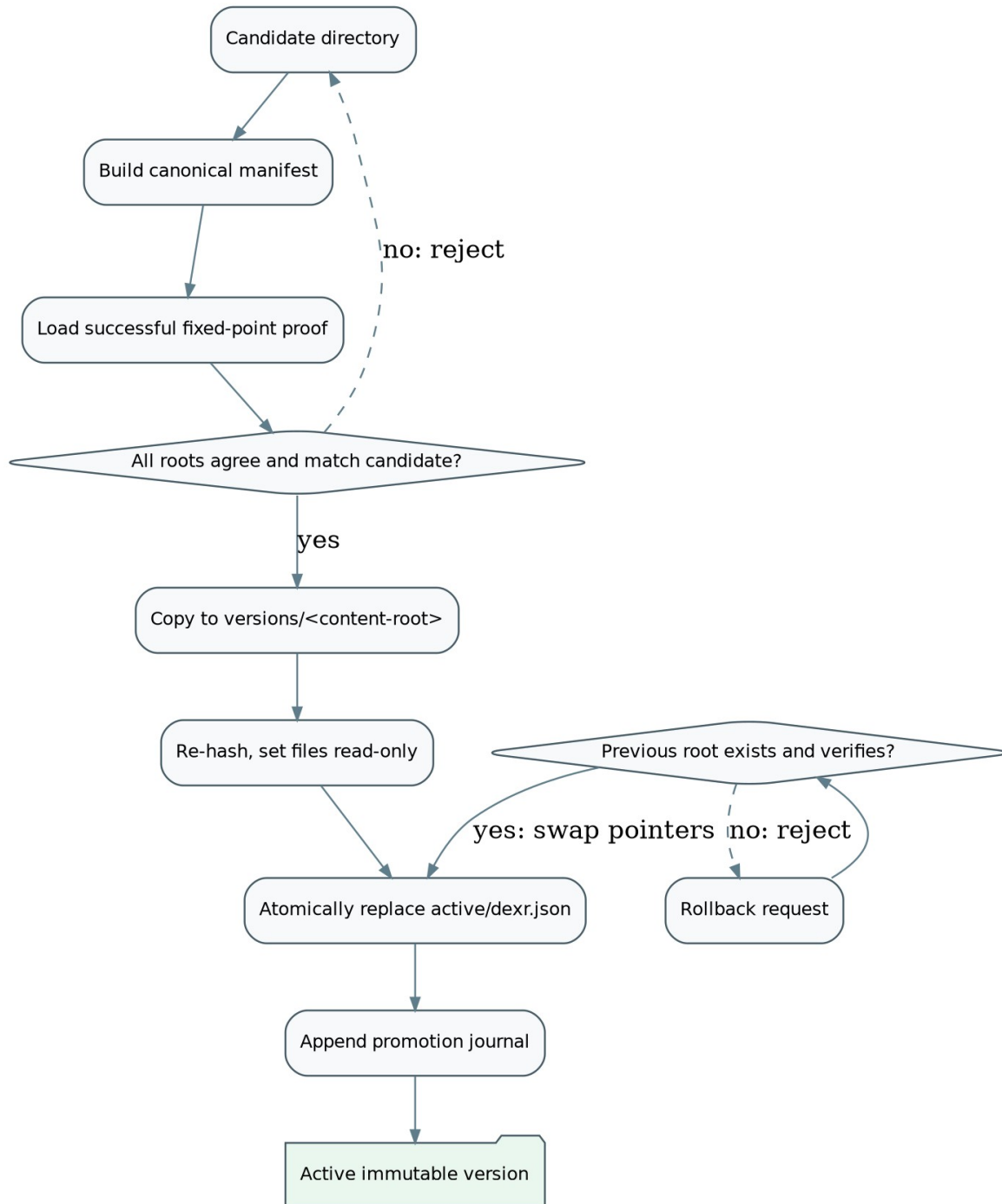
02 Request Lifecycle



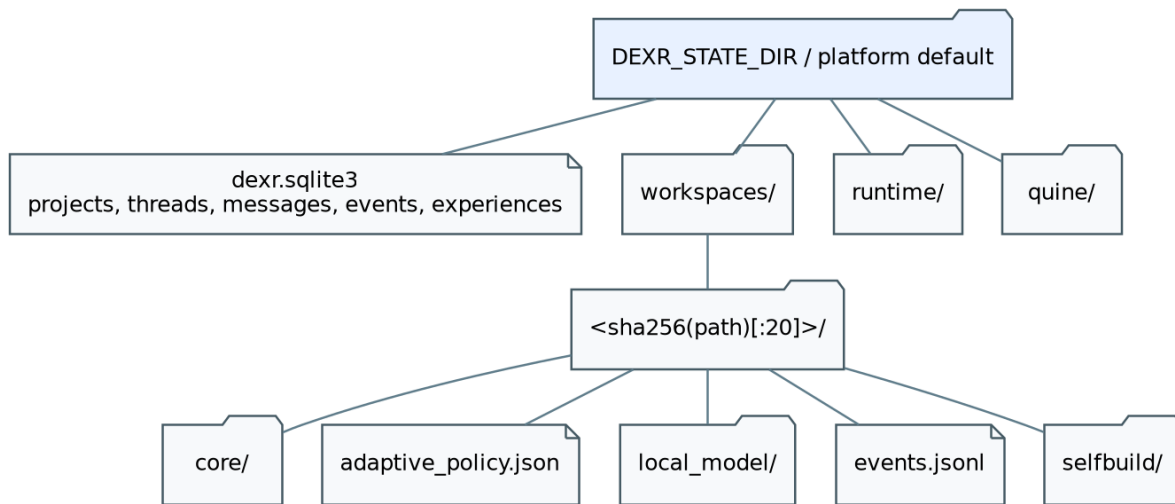
03 Quine Fixed Point



04 Quine Promotion Rollback



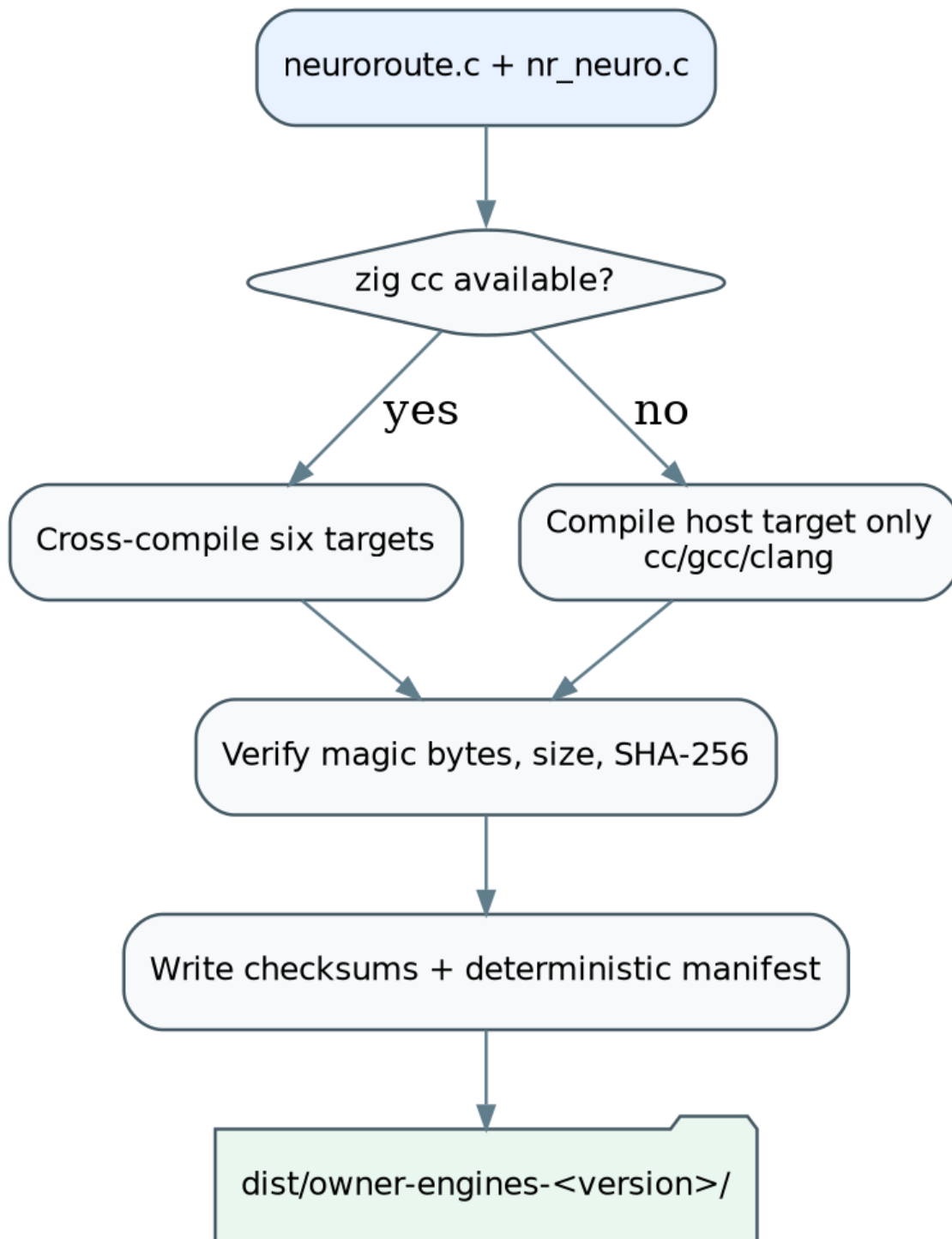
05 State Layout



06 Verification Gates



07 Native Build Matrix



08 Repair Decision Tree

